Saudi Dialect End Of Utterance Project

The objective of this project is to develop a robust AI model capable of detecting "End of Utterance" (EOU) in real-time. This capability is critical for Voice AI Agents, enabling them to understand when a user has finished speaking versus when they are merely pausing or hesitating. The challenge lies in distinguishing between a complete semantic turn (Response) and a mid-sentence pause (Wait), specifically within the context of the **Saudi Dialect**

Step 1: Collecting the dataset:

We are in the most important phase of the project that if it will design and found a great data the result of the model should be very well.

As I start I tried to find some great Saudi dialect dataset to train the model so three ideas come cross my mind.

1. Transcript videos and from social media (YouTube, TikTok, Instagram,…)
2. Try to find public datasets on Hugging Face or Kaggle or any public resource.
3. Create a synthetic data using LLMs for this task but this dataset should be well designed and really reflect the real situation.

The dataset should be conversational dataset and should reflect realistic Arabic dialogue, including features such as natural pauses, incomplete turns, and with emphasis on the Saudi dialect.

So I thought the first option had disadvantages such as:

- It really needs a lot of time to find the most appropriate dataset for this specific purpose. Because our goal is the data should represent a conversation.
- Transcription of these videos will take some time, although we will use a STT model for this but still it needs manual checking.
- Most of the resources will not represent a conversational dataset that represent our final goal (conversational like a call)

So my solution leverage here!

The Smart Hybrid Approach:

I will combine the Second and the Third option (Public Dataset + synthetic data) to create my own dataset this approach solid because it combines them.

1. Public Dataset: AI-Diploma/saudi-dialect-classification-train And Omartificial-Intelligence-Space/saudi-dialect-test-samples: The most suitable dataset I found they are mostly the same but I take the unique of each one:
    a. **Saudi Dialect Focus:** Explicitly contains Najdi, Hijazi, and other local variations, ensuring the model handles local vernacular (e.g., "ياخي", "سم", "أبشر") correctly.
    b. **Diverse Scenarios:** Contains 44 categories covering various domains (Travel, Government, Food), preventing domain-overfitting.
    c. **Conversational Structure (Critical for EOU):** Includes specific classes like Questions, Responses, and Greetings. These represent natural "Turn-Taking" points, providing high-quality positive examples (Label 1) for end-of-turn detection.
    d. **Cleanliness:** The text is relatively clean from extreme noise, making it a reliable "Ground Truth" for complete sentences.



Examples of the dataset:

category string · *classes* 43 values	text string · *lengths* 19 111
Greetings	السلام عليكم ورحمة الله وبركاته، كيف حالكم جميعاً اليوم؟ أتمنى لكم يوماً مباركاً مليئاً بالخير.
Greetings	صباح الخير والسرور على وجوهكم الطيبة في ذا اليوم الجميل، عسى الله يكتب لنا فيه كل توفيق.
Greetings	مساء الخير يا أصدقائي الأعزاء، أتمنى أن تكونوا قد قضيتم يوماً رائعاً وأن ترتاحوا في هذه الأمسية.
Greetings	مرحباً بكم جميعاً في هذا اللقاء المميز الذي طالما انتظرناه بفارغ الصبر، سعدنا بحضوركم.
Responses	ممكن، هو متأكد مية بالمية، نحتاج نبحث ونحلل أكثر قبل ما ناخذ قرار نهائي.
Responses	طيب، راح أسويها بكل سرور وعلى أكمل وجه، اعتبر الموضوع منتهي من اللحظة ذي.
Responses	شكراً جزيلاً على مساعدتك الغالية، لولا الله ثم انت ما قدرت أخلص هالشغلة بالوقت المحدد.
Time	البعد التنازلي للفعالية الكبيرة وين وصل؟ قرب الموعد ولا باقي؟
Time	موعد حجزي في المطعم متى بالضبط؟ أقدر أأجله شوي؟
Time	وش تاريخ اليوم؟ هجري وميلادي؟ وهل هم نفس اليوم؟

2. Synthetic Data: Because ALLaM is a Saudi LLM that train on massively hige Arabic and Saudi Data Corpus I choose it. I really compared it's result with gpt-4o in this task its totally different and ALLaM was really capturing the Saudi Dialect better.
3. for creating a scenario based synthetic data for complete sentences and non-complete

Using Few-Shots prompting and letting him generate 1000 examples per class:

```
def get_eou_generation_prompt(target_label):
    """
    يبني برومبت مخصص مع أمثلة سعودية  وواقعية جداً.
    """

    if target_label == 0:
        task_desc = "توليد جمل سعودية عامية** (Label 0). غير مكتملة** الجملة يجب أن تنقطع فجأة وكأن المتحدث يفكر، يتردد، او يفكر وش يقول."
        examples = """
(لاحظ كيف تنقطع الجملة) أمثلة للجمل المطلوبة:
- "ياخي المشكلة إني يوم رحت لهم قالوا لي..."
- "طيب بخصوص الطلب اللي طلبته أمس هل..."
- "رقم جوالي صفر خمسة خمسة... ثلاثة..."
- "يعني الصراحة ما أدري وش..."
- "لو سمحت أبغى أحجز موعد عند دكتور..."
- "أنا حسابي في البنك الـ..."
"""
    else:
        task_desc = "توليد جمل سعودية عامية** (Label 1). مكتملة المعنى تماماً** الجملة يجب أن تكون واضحة ومنتهية (سؤال كامل، رد، أو طلب)."
        examples = """
أمثلة للجمل المطلوبة (جمل منتهية):
- "متى يفتح الفرع عندكم اليوم؟"
- "خلاص تمام الله يعطيك العافية."
- "لا ما أبغى أي إضافات شكراً."
- "أبغى أرفع شكوى على المندوب."
- "كم سعر التوصيل لحي الملقا؟"
- "إيه نعم صحيح."
"""

    prompt = f"""
أنت كاتب حوارات سعودي محترف.
المهمة: {task_desc}

{examples}
الآن دورك. فكر خطوة بخطوة لكل مثال، ثم اكتب الجملة النهائية فقط.
الشروط:
1. اكتب 5 جمل جديدة مختلفة عن الأمثلة.
2. اللهجة يجب أن تكون بيضاء أو نجدية/حجازية مفهومة.
3. **لا تكتب** أي مقدمات مثل "إليك الجمل" أو "التفكير".
4. اكتب الجملة مباشرة في كل سطر.

ابدأ الكتابة الآن:
"""
```

I really constructed this prompt after many trials and failures this prompt resolved these problems:

a. **Label-Content Mismatch (The "Visual Cue" Issue):**

- **Challenge:** Initial prompts lacked explicit visual markers for incompleteness (like ellipses "...") in the few-shot examples.
- **Impact:** The model generated grammatically complete sentences but incorrectly labeled them as "Incomplete (0)," creating noisy training data.
- **Solution:** Revised the few-shot prompt to include trailing ellipses (e.g., "أبغى أحجز...") in all incomplete examples, forcing the model to generate text that visually reflects an interruption.
- **These "…" of course will be deleted in further step.**

b. **Formatting Noise:**

- **Challenge:** The LLM treated the task as writing a dialogue script, adding speaker prefixes like **(Ahmed: ...)** which are irrelevant to the audio transcription task.

- **Solution:** Implemented strict negative constraints in the system prompt (e.g., "Do not write names," "Output raw text only") to ensure clean input for the BERT model.

Step 2: Data Preprocessing:

To ensure high data quality and avoid model confusion, I implemented a strict and deterministic preprocessing pipeline:

1. **Advanced Text Normalization:**
    - **Objective:** To simulate raw ASR (STT output) output and unify spelling variations.
    - **Technique:**
        - **Normalization:** Unified Alef forms (أ, إ, آ to ا) and Ya forms (ى to ي) to handle inconsistency in writing styles.
        - **Cleaning:** Removed all diacritics (Tashkeel) and punctuation marks (., ?, !) as they are not present in real-time audio streams.
2. Label Engineering:

    I used a structured approach to generate "Incomplete" samples (Label 0) from the public datasets:

    - **Label 1 (Complete):** The original normalized sentence.
    - **Label 0 (Incomplete):** Generated by removing the **last word** and the **last two words** of the sentence (only if the sentence length > 3 words).
    - *Justification:* This teaches the model that a sentence is "incomplete" specifically when the final stopping keyword is missing, creating a sharp decision boundary.
3. Synthetic Injection:

    Integrated 2136 high-quality synthetic examples generated by ALLaM (1089 complete, 1047 incomplete) <The used prompt is in the pervious section> to cover edge cases like hesitation and digit dictation not found in public datasets.

Examples:

text	label
ترى للحين ما حجزنا الطاولة	1
انت متأكد إنك	0
كيف كان الاجتماع الصباحي اليوم	1

4. **Data Splitting Strategy (<u>After combining Real and Synthetic Dataset</u>):**
    - **Split First:** The dataset was split into Train (70%), Validation (15%), and Test (15%) **before** balancing.
    - **Held-Out Test Set:** The Test set was kept purely "original" (no duplication) to serve as a fair final evaluation.
5. **Class Balancing (Train Set Only):**
    - Applied **Upsampling** only on the Training set to enforce a 50/50 balance between COMPLETE and INCOMPLETE classes, ensuring the model does not develop a bias toward the majority class.

Table 1: Final Dataset Splits for EOU Model Training

Dataset Split	Samples	Description
Training Set (Balanced)	4,976	Used for model learning; upsampled to ensure a 50/50 class balance.
Validation Set	886	Used for benchmarking and model checkpoint selection during training.
Test Set (Held-out)	889	Kept completely unseen for the final unbiased performance evaluation.
Total	**6,751**	

Step 3: Model Selection & Benchmarking:

Why BERT Architecture?

For the specific task of End-of-Utterance (EOU) detection, I chose an **Encoder-only architecture (BERT)** rather than a Generative Decoder model (like GPT/Llama) for three critical engineering reasons:

1. **Latency & Real-Time Performance:**
    - Voice agents require decision-making in milliseconds to avoid latency in the conversation.
    - BERT models are lightweight (~110M parameters) compared to LLMs (7B+ parameters), making them significantly faster for real-time inference on the edge or cloud.
2. **Deep Contextual Understanding:**
    - BERT's bidirectional attention mechanism allows it to analyze the syntactic and semantic structure of the sentence deeply. This is vital for detecting subtle "incompleteness" cues in the Saudi dialect, such as hanging prepositions or specific hesitation patterns.

Choosing the right foundation model is critical for Arabic dialects. I conducted a comparative experiment (Benchmark) to find the best performer.

1. Candidate Models:

I selected three top-tier Arabic BERT models to test:

- **UBC-NLP/MARBERTv2:** A powerful general Arabic model trained on 1 billion tweets. It is known for handling various dialects well.
- **faisalq/SaudiBERT:** A specialized model pre-trained specifically on a large corpus of Saudi text.
- **CAMeL-Lab/bert-base-arabic-camelbert-da:** A model optimized for Dialectal Arabic (DA).

The Experiment:

- I trained all three models on the same balanced dataset for **2 Epochs**.
- I compared them based on **F1 Score** (because balancing precision and recall is vital for this task) and **Accuracy**.

Results & Decision:

- **SaudiBERT** achieved the highest F1 Score **(~79.32%**), establishing a strong baseline despite the inherent ambiguity of text-only EOU detection.
- **MARBERTv2** came in second place with close results (**~76.52%**).
- **Decision:** I selected **SaudiBERT** as the final model.
    - **Justification:** It demonstrated a better understanding of local Saudi idioms and give me better result on accuracy and F1 Score.

Table 2: Model Selection Benchmark Results (2 Epochs)

Model Name	Accuracy	F1 Score
faisalq/SaudiBERT	**82.17%**	**79.32%**
UBC-NLP/MARBERTv2	79.01%	76.52%
CAMeL-Lab/bert-base-arabic-camelbert-da	72.91%	68.09%

Step 4: Model Fine-tuning:

After selecting *SaudiBERT*, I moved to the final "Production Training" phase to maximize performance.

Training Configuration:

- **Hyperparameters:**
    - **Learning Rate:** 1e-5 (A low learning rate was used to fine-tune the model carefully without destroying its pre-trained knowledge).

- **Batch Size:** 16 (To provide stable updates).
- **Epochs:** 3 to 4 (To prevent overfitting).
- **Metric:** Optimized for **F1 Score**.

- **Strategy:** I used the load_best_model_at_end=True feature. This ensures that even if the model's performance drops in the final epoch, the system will automatically save the best version based on the validation set.

Final Results (On Held-Out Test Set):

Table 3: Final Evaluation Results on Held-Out Test Set

Metric	Score
Accuracy	80.76%
F1 Score	77.47%

Step 5: Model Deployment & SDK:

A raw model file is not enough for a real-time application. I built a complete deployment package to integrate the model with LiveKit.

1. The Python SDK (saudi_eou.py)

I created a custom Python class to wrap the model. This SDK handles more than just prediction; it ensures reliability:

- **Text Normalization:** It automatically cleans incoming text (unifying Alef/Ya and removing punctuation) so the model sees the text exactly as it was trained.
- **Confidence Threshold:** The model only triggers a "REPLY" signal if the confidence score is above **0.5**, ensuring the agent does not interrupt unless it is sure.

2. LiveKit Integration

- I developed an adapter script (agent.py) that connects to the LiveKit server.
- It intercepts the transcription stream from the Speech-to-Text (STT) engine.
- It sends the text to my Saudi EOU model in real-time.
- If the model predicts "REPLY," the agent stops listening and generates a response.

Conclusion

This project successfully solved the "Turn-Taking" problem for Saudi Voice AI. By combining a **Smart Hybrid Dataset** (Real + ALLaM Synthetic) with a specialized **SaudiBERT** model, I achieved

a system (77.47% F1 Score). The final deliverable is not just a model, but a full SDK ready for production use.

Links:

Hugging Face Model Link: SuperSl6/saudi-eou-model-final

Hugging Face Space Link: space SuperSl6/saudi-eou-model-final

Hugging Face Data Link: SuperSl6/saudi-eou-dataset-final